

01 intro

cumulative outputs

→ need to look at all/multiple samples together to check if successful or not

Objectives

transform data, explain variation, reduce entropy, likelihood, distances in clusters or between data points

for analysing, explore, finding structure, visualize or compressing data
express data, generate new knowledge

Goal

select model with

- best generalization performance i.e. best performance on future data

often only used for analysing training data

model selection=training=learning (synonyms)

find model best explaining/approximating training set

#overfitting also problem of unsupervised learning

model adapting characteristics like noise, outliers or labelling errors (but there are no labels in unsupervised)

some methods

Unsupervised Methods:

- principal component analysis
- independent component analysis
- factor analysis
- projection pursuit
- k-means clustering
- hierarchical clustering
- mixture models: Gaussian mixtures
- self-organizing maps
- kernel density estimation
- hidden Markov models
- Markov networks (Markov random fields)
- restricted Boltzmann machines
- neural network: auto-associators, unsupervised deep nets

Projection methods:

map vectors to another space

create new representation of vectors

e.g. down-projection with keeping neighbourhoods (e.g. for visualization)

finding structure

generative models: create model from data, data produced from model should match real world

downprojection: PCA (with orthogonal dimensions)

ICA: components should be independent of each other

factor analysis: PCA with noise

density estimation: build function producing same data in same places with same density

clustering

data might overlap

1 or 2 clusters?

cluster might be inside other cluster

humans look at connected elements

e.g. data points that can be connected without gaps->same cluster
computer often tries to find cluster center and elements near cluster center->in cluster

Blind source separation

2 people speaking independently

2 microphones

different people have different distances to microphones

basically multiple sounds with different volumes in single source

separate voices (e.g. using ICA)

ICA can also be used on e.g. images

with many edge detectors->find actual edges

types of unsupervised models

Parametric models

parameter vector represents model

model often represented by 2 parameter vectors (mapping to same model)

e.g. NNs, SVMs or factor analysis are parametric models

learning->learn parameters

problem: same function could have different representations

model complexity can only be described using parameters

Nonparametric models

locally constant/fixed model

e.g. k-NNs, kernel density estimation (choose kernel/density function, not parameters), decision trees (determined by data, no parameters)

rules/hyperparameters need to be selected apriori (e.g. k for k-NN, kernel with or splitting rules)

descriptive models

=projection methods

different description of data with some properties

e.g. PCA, ICA

generative models

should produce data similar to real-world data
(typically) requires random components for producing data
typically includes prior knowledge
some parameters are unknown → learn

02 basic terms and concepts

Unsupervised categories

generative

density estimation, hidden markov models
objectives: max likelihood that real world generates something similar

recording or descriptive frameworks

objectives like max variance, orthogonality, independency, max entropy (non-noise)
data pushed into model, produces distribution
should look like target distribution (e.g. gaussian)

projection methods

principle component analysis (PCA)

independent component analysis

- projects into space with statistically independent components

- results in factorial distribution

- easier optimizations with factorial distributions (e.g. max entropy)

or try to find deviations from default values/go away from default values (e.g. try to create non-gaussianity)

generative models

generation is probabilistic

parameter estimation

model class for real data known, need to find true parameters for it
generative models estimate true parameters based on known model class

loss: difference between true and estimated parameters
estimator is estimated with expected loss
average over all data

theoretical concepts

training data $\mathbf{x}$ is a vector
true parameter vector $\mathbf{w}$
parameter estimate: $\hat{w}$ with caret
estimates have caret on top

unbiased estimator: matches parameter perfectly on average (neither too high nor too low)

→over multiple data points

bias: difference between estimate and true parameter

variance: deviation from average

mean squared error: (expected) squared difference between estimate and true values

averaged over multiple data points

→minimized

$$mse(\hat{w}) = var(\hat{w}) + b^2(\hat{w})$$

mse... mean squared error

b... bias

var... variance

often (when creating estimators): get estimator with no bias, then reduce variance as much as possible

maximum likelihood

one of major objectives in unsupervised learning

asymptotically efficient and unbiased

asymptotically efficient→lowest variance with enough data points

does everything right with minimum variance for enough data

makes optimal use of data

Likelihood

likelihood of data set is probability to produce specific dataset

assumes independent and identically distributed data

probabilities get low quickly

→negative log likelihood

instead of maximizing likelihood→minimize negative log-likelihood

properties

invariant under parameter changes

known solution, change parameters $\rightarrow$ new solution can be derived from old solution

asymptotically unbiased and efficient $\rightarrow$ asymptotically optimal

asymptotically consistent

more data $\rightarrow$ gets closer and closer to real values

actually approaches true value with enough training data

for getting likelihood $\rightarrow$ requires to find all input

$\rightarrow$ needs to derive reverse model

need to integrate over all inputs

EM

for maximizing likelihood

problem: hard to determine which hidden variables can produce classes

replace with simpler distribution over random variables

alternating E and M step

E

maximize density Q of that distribution

tightens bound (or even equal)

increases lower bound

using parameter w

M

maximize parameter for lower bound w

E-step:

$$Q_{k+1} = \arg \max_Q \mathcal{F}(Q, w_k)$$

M-step:

$$w_{k+1} = \arg \max_w \mathcal{F}(Q_{k+1}, w)$$

both steps tighten bound
E-Step doesn't change parameters
used for HMMs, mixture of gaussians, factor analysis, ICA, ...

Maximum Entropy

max entropy probability distributions
e.g. Gauss or exponential distribution
class of distributions, look for distr with max entropy
max entropy=minimizing prior assumptions on distributions
e.g. not much information/assumptions in gauss distribution
physical systems->like max entropy
most likely observed solution
many connections to physics/statistical mechanics

principle of maximum entropy

entropy

$$H = - \sum_{k \geq 1} p_k \log p_k$$

zero classes:

$$p_k \log p_k = 0 \text{ for } p_k = 0$$

p_k - probability of being in class k
k...classes ==> sum of classes

normal distribution is max entropy distribution over all distribution with given mean and stdev
uniform distribution is max entropy distribution of all distributions in fixed interval
exponential distribution is max entropy distribution of all distribution with fixed mean which go from 0 to infinity

Some classes of distributions might not have max entropy distribution
 e.g. because distribution not bounded/go to infinity
 e.g. distributions with mean $\rightarrow$ arbitrary large entropy
 e.g. entropies bounded from above but bound is not part of distribution

constraints

some moments of distribution need to be fixed
 probabilities need to be ≥ 0 (required for log) and sum up to 1

$$\sum_{i=1}^n p(x_i) f_k(x_i) = F_k \quad k = 1, \dots, m$$

$$\sum_{i=1}^n p(x_i) = 1$$

Solution (with constraints)

Gibbs distribution
 goal: maximize entropy
 exponential distribution
 $m \dots$ fixed moments

$$p(x_i) = \frac{1}{Z(\lambda_1, \dots, \lambda_m)} \exp(\lambda_1 f_1(x_i) + \dots + \lambda_m f_m(x_i))$$

normalizing constant Z
 guarantees second constraint
 Normalizing constant: partition function

$$Z(\lambda_1, \dots, \lambda_m) = \sum_{i=1}^n \exp(\lambda_1 f_1(x_i) + \dots + \lambda_m f_m(x_i))$$

lambdas: Lagrange multipliers

$$F_k = \frac{\partial}{\partial \lambda_k} \log Z(\lambda_1, \dots, \lambda_m)$$

obtained from first constraint

03 Principal Component Analysis

preprocessing/downprojection

also “Karhunen-Loève transform” or “Hotelling transform”

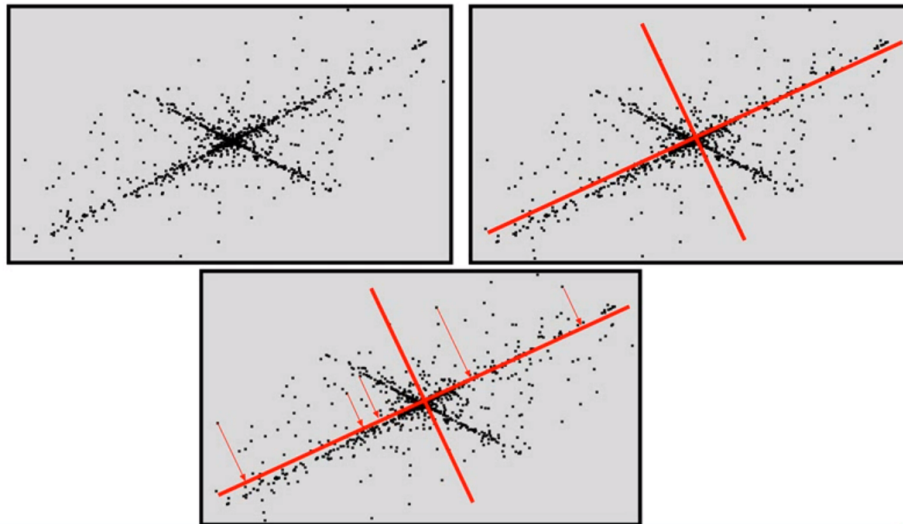
only changes coordinate system, data points stay the same

“look at data from different angle”

first coordinate $\rightarrow$ largest variance

other coordinates orthogonal, 2. coord $\rightarrow$ 2. largest variance, ...

then project onto “principal component” (coordinate axis)



advantage

only use most important coordinates/components of coordinates system
projection to 2D or 3D $\rightarrow$ visualization

data is set of vectors $\{x_1, x_2, \dots, x_n\}$

might be summarized by data matrix $X = (x_1, x_2, \dots, x_n) \in \mathbb{R}^{n \times n}$

rows $\rightarrow$ observations

columns $\rightarrow$ features

could be transposed

PCA requires mean of 0

normalize first

calculate mean per feature per component, subtract mean

using sample covariance matrix

=empirical covariance matrix

based on samples

$$C \in \mathbb{R}^{m \times m}$$

features $\times$ features

entry

$$C_{st} = \frac{1}{n} \sum_{i=1}^n x_{is} x_{it} \text{ (position s,t)}$$

in vector notation

$$C = \frac{1}{n} X^T X = \frac{1}{n} U D_m U^T$$

$X^T X$ is positive definite

can be transposed in U and U^T

D_m contains all eigenvalues of $X^T X$

all ≥ 0 , sorted decreasingly

called eigendecomposition or spectral decomposition of C

column vectors $u_i = [U]_i \rightarrow$ eigenvectors (principal components)

first eigenvector $\leftrightarrow$ first eigenvalue etc

assume vectors are linear independent

$\rightarrow C$ has full rank

$\rightarrow$ positive eigenvalues

otherwise might be 0

singular value decomposition

alternative

directly decomposes matrix X into VDU^T

$D \in \mathbb{R}^{n \times m}$ diagonal matrix

positive entries "singular values", sorted decreasingly

$U \in \mathbb{R}^{m \times m}$ and $V \in \mathbb{R}^{n \times n}$

first entry of $U/V \leftrightarrow$ max value of D

advantage

$$Y = XU = VD$$

$\Rightarrow$ values are/can be obtained directly

PCA projection directly recovered

$Y \dots$ new representation of data

single data point: $y = U^T x$

outer product representation

express X with outer product

$$X = \sum_{i=1}^m D_{ii} v_i u_i^T = \sum_{j=1}^m y_j u_j^T$$

$$y = D_{ii} v_i$$

iterative methods

one update at one time

using “Oja’s Rule”

current projection $t = u^T x$

$u^{new} = u + \eta(tx - t^2 u)$

η ... learning rate

initially random

update until convergence

u converges to largest principal component

properties of PCA

first principal component $\rightarrow$ max variance

- unique solution up to signs
if eigenvalues are different/unique
so sign/direction can be switched
everything else unique
signs are not deterministic
- first l principal components span l -dimensional space with *maximal variance*
- orthogonal principal components
- projections have zero means
- projections on principal components are uncorrelated/orthogonal
- sample variance of k -th projection = k -th eigenvalue of sample covariance matrix
- principal components ranked decreasingly by eigenvalues
- first l principal components minimize mean squared error

Kernel PCA

nonlinear PCA/kernel PCA

first map data vectors in high-dimensional space

some nonlinear mapping
normal linear PCA in that space

kernel gives mapping to high dimensional space

$$\mathbf{x} \mapsto \Phi(\mathbf{x})$$

assume data is centered in feature space

$$\sum_{i=1}^n \Phi(\mathbf{x}_i) = \mathbf{0}$$

look at covariance matrix in feature space

$$C = \frac{1}{n} \sum_{i=1}^n \Phi(x_i) \Phi^T(x_i)$$

phi is not explicitly known

gram matrix known, inner product and dot products can be calculated

$$K : K_{ij} \Phi^T(x_j) \Phi(x_i)$$

gram matrix $\rightarrow$ matrix of all dot products

only have

$$K_{ij} = k(x_i, x_j) = \Phi^T(x_j) \Phi(x_i)$$

is dot product in some high-dimensional space

need to calculate outer product

actually not covariance matrix but eigenvectors

PCA $\rightarrow$ maximize variance

map n data points into high-dimensional space

high dimensional space is restricted to span of vectors mapped to it

$$\text{span}\{\Phi(x_1), \dots, \Phi(x_n)\}$$

project left and right side of eigenvalue equation into high-dimensional space

both left and right side can be expressed by dot products

$$n\lambda K\alpha = K^2\alpha$$

assume Gram matrix has full rank $\rightarrow n\lambda\alpha = K\alpha$

compute Gram matrix and eigenvectors

using kernel function

need to normalize eigenvectors to length 1

$\alpha \dots$ eigenvectors..?

just determine eigenvectors of Gram matrix, normalize by $\alpha_i^{new} = \frac{\alpha_i}{\|\alpha\| \sqrt{n\lambda}}$

new data point

need data centering

done in feature space/before eigenvalue decomposition of Gram matrix

Gram matrix

$$\begin{aligned}
& \left(\Phi(\mathbf{x}_i) - \frac{1}{n} \sum_{t=1}^n \Phi(\mathbf{x}_t) \right)^T \left(\Phi(\mathbf{x}_j) - \frac{1}{n} \sum_{t=1}^n \Phi(\mathbf{x}_t) \right) \\
& \left(\Phi(\mathbf{x}_i) - \frac{1}{n} \sum_{t=1}^n \Phi(\mathbf{x}_t) \right)^T \left(\Phi(\mathbf{x}_j) - \frac{1}{n} \sum_{t=1}^n \Phi(\mathbf{x}_t) \right) = \\
& \Phi^T(\mathbf{x}_i) \Phi(\mathbf{x}_j) - \frac{1}{n} \sum_{t=1}^n \Phi^T(\mathbf{x}_t) \Phi(\mathbf{x}_j) - \frac{1}{n} \sum_{t=1}^n \Phi^T(\mathbf{x}_i) \Phi(\mathbf{x}_t) + \\
& \frac{1}{n^2} \sum_{(s,t)=(1,1)}^{(n,n)} \Phi^T(\mathbf{x}_s) \Phi(\mathbf{x}_t)
\end{aligned}$$

basically center data in kernel space

all terms can be expressed in Gram matrix

Kernel PCA algorithm

Given: gram matrix $\mathbf{K}$ with $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$

Centering

center the Gram matrix $\mathbf{K} \leftarrow \mathbf{K} - \frac{1}{n} \mathbf{K} \mathbf{1} \mathbf{1}^T - \frac{1}{n} \mathbf{1} \mathbf{1}^T \mathbf{K} + \frac{1}{n^2} (\mathbf{1}^T \mathbf{K} \mathbf{1}) \mathbf{1} \mathbf{1}^T$

Eigenvalues

compute eigenvectors α and eigenvalues λ of the Gram matrix $\mathbf{K}$

Normalization

normalize eigenvectors $\alpha \quad \alpha_i^{\text{new}} = \frac{\alpha_i}{\|\alpha\| \sqrt{n \lambda}}$

Projection of a new vector

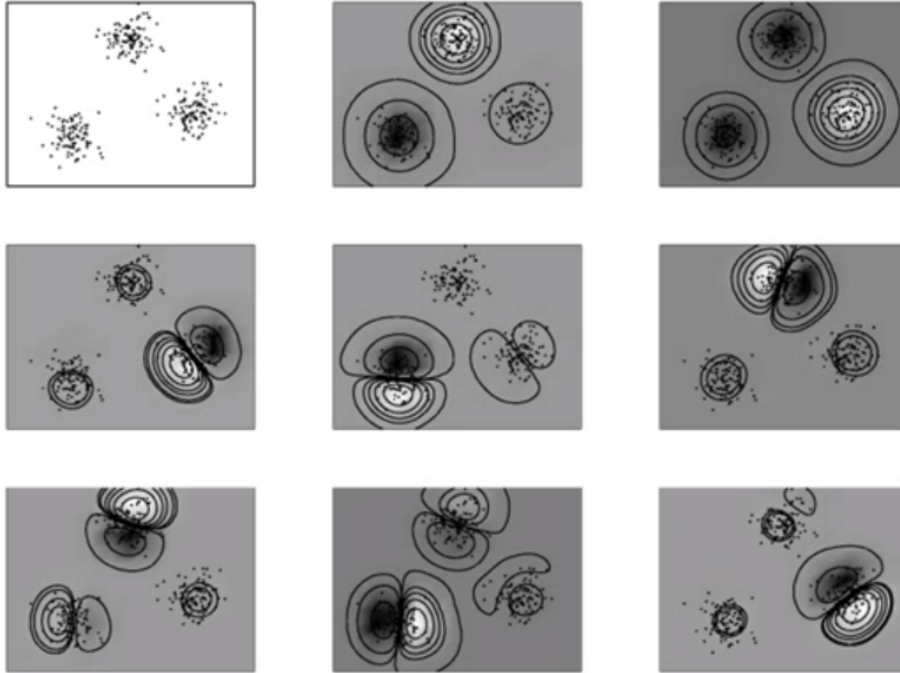
project a new vector $\mathbf{x}$ onto eigenvectors by center and project it

$$\mathbf{k}(\mathbf{x}, \cdot) - \frac{1}{n} \mathbf{K} \mathbf{1} - \frac{1}{n} \mathbf{1}^T \mathbf{k}(\mathbf{x}, \cdot) \mathbf{1} + \frac{1}{n^2} (\mathbf{1}^T \mathbf{K} \mathbf{1}) \mathbf{1}$$

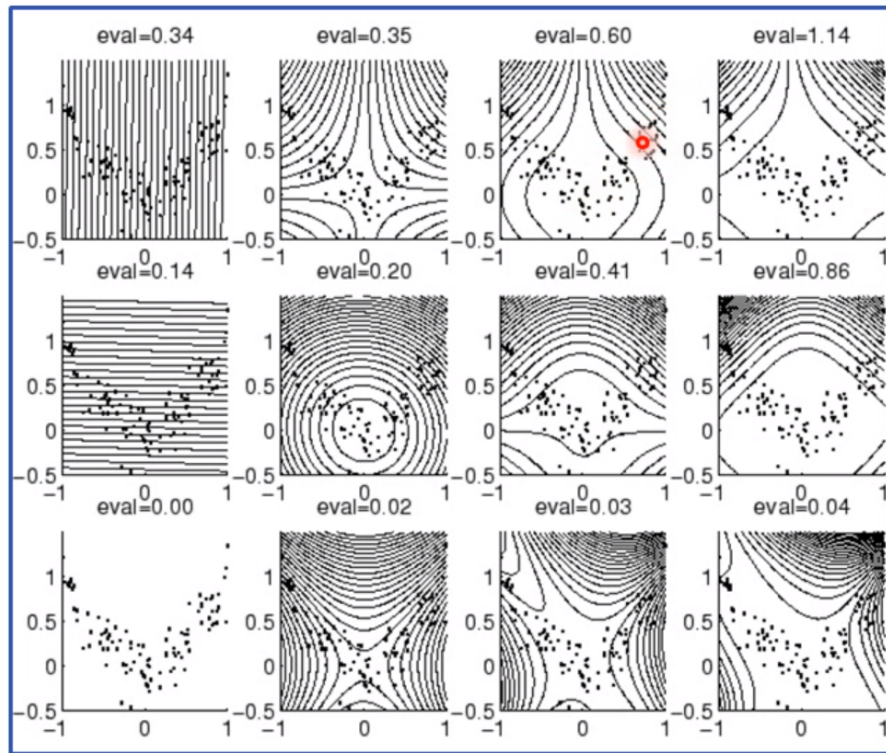
$$\mathbf{w}^T \Phi(\mathbf{x}) = \sum_{i=1}^n \alpha_i \Phi^T(\mathbf{x}_i) \Phi(\mathbf{x}) = \sum_{i=1}^n \alpha_i k(\mathbf{x}_i, \mathbf{x})$$

kernel PCA works with clustered data

principal components:



first separate clusters, then divide in halves, then orthogonal division



data bottom left

isolines intersect parabola orthogonally

in principal components with high eigenvalues

feature map never actually calculated but instead reformulated using kernel
make linear method nonlinear

04 Independent Component Analysis

checks for statistically independent components

assume all (target) components are independent $\rightarrow$ probabilities can be multiplied

$$p(y) = \prod_{j=1}^l p(y_i)$$

$$x = Uy$$

U is $l \times l$ matrix

point mapped to new type of features

$$y = Wx$$

new features should be independent

$$W = U^{-1}$$

need $Y^T Y = D_m$ to be diagonal matrix (only elements on diagonal are 0)
should be decorrelated *and* statistically independent

matrix decomposition problem

used for e.g. separating voices

2 people, 2 microphones (at different positions)

not talking to each other, different languages

different voices $\rightarrow$ different volumes

both microphones pick up mix of speeches

separate them

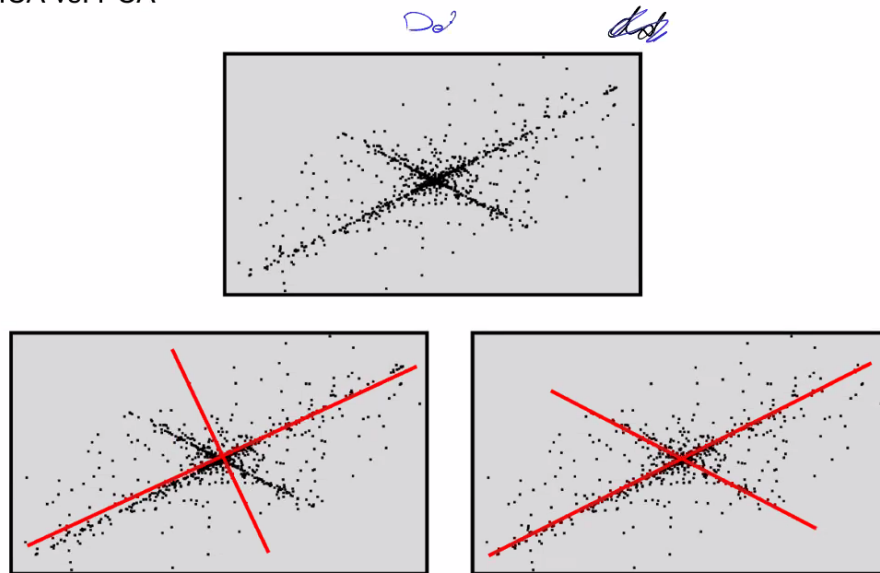
ICA is for separating/deconstructing mixed features

if people talked to each other, components would be dependent

should be independent

reconstructed sources could be ordered differently

ICA vs. PCA



left: PCA, right: ICA

PCA components are orthogonal

properties

solution from ICA is not unique

for $x = Uy \rightarrow$ can squeeze in $x = UP^{-1}Py$

changes solution

but unique up to permutational scaling

Darmois' Theorem

as many random variables as components

$$x_1 = \sum_{j=1}^m a_j y_j, x_2 = \sum_{j=1}^m b_j y_j$$

if random variables are independent then all variables where $a_j b_j \neq 0$ are gaussian

ICA doesn't work well with gaussian distributions

if sources are gaussian $\rightarrow$ cannot be identified

ICA assumes non-gaussian sources

ICA solution cannot mix independent components

different solutions unique up to permutation or scaling

doesn't work for ICA

assumptions

(all but one) sources are non-Gaussian

$l \geq m$ for as many observations as sources

for simplicity, $l = m$ (number of input and target features)

U has full rank l

one component can be gaussian, not two

measure independence

optimize for independent components

can be measured by $p(y_i | y_1, \dots, y_{i-1}, y_{i+1}, \dots, y_l) = p(y_i)$

criteria for independence

- mutual information between components
- measure non-Gaussianity

mutual information

sum over entropy measure how unordered some information is

$$I(y_1, \dots, y_l) = \sum_{j=1}^l H(y_j) - H(y)$$

H denotes entropy, $H(a) = - \int p(a) \ln p(a) da = -E[\ln p(a)]$ can be rewritten as expected value of log likelihood

$$I(y_1, \dots, y_n) = \sum_{j=1}^m H(y_j) - H(x) - \ln |W|$$

$|W|$ is absolute value of determinant

$$p(y) = \frac{p(x)}{|W|}$$

because of "Change of variables theorem"

non-Gaussianity

use negentropy

assume gaussian, calculate entropy, calculate difference

$$J(y) = H(y_{gauss}) - H(y)$$

estimation of negentropy is difficult

==> use kurtosis instead of entropy

kurtosis

$E(x)$ -> mean

second moment -> variance

$$\kappa_1 = E(x) = 0 \text{ mean}$$

$$\kappa_2 = E(x^2) \text{ variance}$$

$$\kappa_3 = E(x^3) \text{ skewness}$$

$$\kappa_4 = E(x^4) - 3(E(x^2))^2 \text{ kurtosis}$$

how "heavy" the tails of gaussians are

~ likelihood of outliers

$$\text{kurtosis} = \frac{\kappa_4}{\kappa_2^2}$$

kurtosis should be maximized for supergaussians

mix of random variables -> always gets closer to gaussian (central limit theorem)

-> not good for ICA

maximizing kurtosis ~ maximizing sparseness

sparseness ~ many values close to 0

kurtosis is affected by outliers/not robust

contrast functions

also for measuring independence

compare against standard gaussian

use gaussian as normalization constant

not as affected by outliers

whitening -> decorrelating

rotating -> for independency

$$\text{whitened data: } Y^T Y = I, X = Y^T, Y = XU^{-T}$$

first "sphere" data

data points are put on surface of sphere

$$\text{take identity matrix, write as } I = C^{-\frac{1}{2}} \frac{1}{n} X^T X C^{-\frac{1}{2}}$$

C -> covariance matrix

then need to get inverse square root matrix

plug in $X = YU^T$
 $x^T x = I$

$\hat{U} = \frac{1}{\sqrt{n}} C^{\frac{-1}{2}} U$ is orthogonal, $C^{\frac{-1}{2}}$ is symmetric $\hat{U}^T$ is orthogonal
 first whiten data then rotate data, test different rotations, check kurtosis of y s,
 change rotation

INFOMAX

minimizes mutual information
 function $g \rightarrow$ transformation
 ~activation function in NNs, e.g. tanh

Change of variable theorem

distribution with inner distribution is outer distribution multiplied with
 absolute determinant of jakobian distribution with inner distribution

$$p(\underbrace{g(y)}_{\text{Jacobian}}) = p(x) \left| \underbrace{\frac{\partial g(y)}{\partial y} \frac{\partial y}{\partial x}}_{\text{Jacobian}} \right|^{-1}$$

COVT *glet*

infomax uses gradient descent for finding W

gradient descent

$$w^{new} = w^{old} - \eta 2g(y)x^T$$

continue until convergence

improvement $\rightarrow$ use natural gradient descent/Fisher information matrix

multiplies gradient with inverse of fisher information matrix

removes matrix inversion $\rightarrow$ performance

easier update rules without matrix inversion

~ Newton Rapson method for optimization

EASI

“equivariant adaptive separation via independence”

FastICA

$$\underline{w}^{new} = E(x g(w^T x)) - E(g'(w^T x)) w$$

most popular ICA algorithm

whitening and rotation
 gradient descent for update rule
 extracts single component, then re-run and decorrelate
 project onto found components subtract already found components (rows of W matrix)

ICA can be generalized to generative approach

ICA extensions:

- generative approach
- sub-Gaussian distributions with specific assumptions *l-m*
- non-linear extensions which are often not unique
- overcomplete basis more sources than observations $l > m$
- fewer sources than observations $l < m$

ICA vs. PCA:

independent component analysis	principal component analysis
causes of the data	geometrical abstractions
statistical independent	decorrelated (orthogonal)
explain super-Gaussians	explain all variance
scale invariant	not scale invariant
unique up to scale and permutation	unique
assume super-Gauss	no assumptions
no ranking	ranked by eigenvalues

ICA components cannot be ranked, sources need to be non-gaussian
 only works for linear mixing/assumes linear mixing

Deep Infomax → with NNs, can recover more complex mixings

statistic independence for ICA, noncorrelation for PCA

X has as many rows as sources, as many columns as features

whitening → set variance in both directions to be similar to original data

rotation → rotate result so it is similar to original data

solution similar to original but not exactly

permutation and scaling

checking for too many independent components

can result in components being split up in subgroups

can make result worse

optimization in multiple dimensions

all eigenvalues $< 0 \implies \max$
all eigenvalues $> 0 \implies \min$

05 Factor Analysis

data matrix X

2 kind of sources, linearly mapped

and also some noise

$$X = YU^T + \Upsilon$$

also assume some noise covariance

matrix decomposition into signal and noise covariance

assume noise is gaussian

explaining variance/variability

Why are observations different from each other

using latent/hidden variables (factors) explaining correlations and noise

generative model

similar to ICA..?

assumes linear dependency of components of components (factors)

factors $\rightarrow Y$

describes variability in observations

using noise and latent factors

noise can only appear within features

no covariances in noise

covariance explained by factors

component-wise noise explained by additive noise term

assumes factors and noise are gaussian

$$\underset{\text{signal}}{\underbrace{x = Uy}} + \underbrace{\epsilon}_{\text{noise}}$$

- $x \in \mathbb{R}^m$ are (centered) observables
- $y \in \mathbb{R}^l \sim \mathcal{N}(\mathbf{0}, I)$ are factors
- $\epsilon \in \mathbb{R}^m \sim \mathcal{N}(\mathbf{0}, \Psi)$ are noise, where Ψ is diagonal
- $U \in \mathbb{R}^{m \times l}$ are factor loadings

$$y \sim \mathcal{N}(0, I)$$

ϵ just changes mean

assume data x is produced by

signal term y

mapped by mapping U (to determine)

some independent/additive noise

m -dimensional space for observables

l factors, to choose

max m , typically much smaller

assume factors y are distributed by standard gaussian

noise term also assumed to be gaussian, centered but with covariance matrix Ψ

Ψ is diagonal

covariances are 0

$$U \in \mathbb{R}^{m \times l}$$

factor loading matrix

maps input to latent factors

conclusions

conditional distribution $x|y$ must also be gaussian

$$x|y \sim \mathcal{N}(Uy, \Psi)$$

because sum of random variables, one gaussian, one linear combination of gaussian

noise is responsible for randomness in conditional distribution

model params U and Ψ (to optimize)

Ψ is diagonal

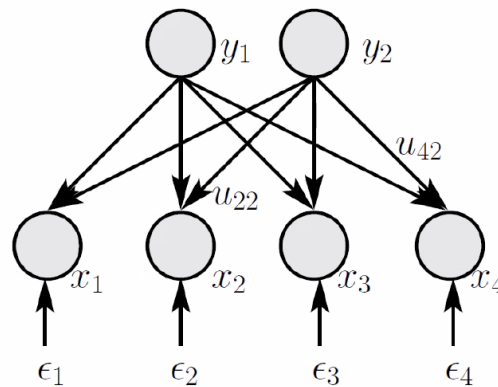
factors must explain all correlations in x

factor y

loading matrix U

observation x

noise ϵ



2 factors, connections with weights u

4dimensional observation space

also includes random component-wise noise

expected value

$$\mathbb{E}[x] = 0$$

$$\mathbb{E}[xx^\top] = UU^\top + \Phi$$

$$x \sim \mathcal{N}(0, UU^\top + \Phi)$$

factor analysis is expectation maximization

observables x

latent variables y

model parameters U, Ψ

expectation $\sim$ expected value

alternate optimizing y and U, Ψ

□ E-step: compute $\mathbb{E}_{y|x}[\log p(x, y)]$ for given U, Ψ

□ M-step: update $U, \Psi = \arg \max_{U, \Psi} \mathbb{E}_{y|x}[\log p(x, y)]$

E-step

trace operator tr

takes diagonal matrix

sum up diagonal

multivariate gaussian

decompose vectors in groups

also decompose covariance matrices

$$\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \sim \mathcal{N} \left(\begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix}, \begin{pmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{pmatrix} \right)$$

conditional distribution also gaussian

$$x_1 \mid x_2 \sim \mathcal{N}(\bar{\mu}, \bar{\Sigma})$$

$$\bar{\mu} = \mu_1 + \Sigma_{12}\Sigma_{22}^{-1}(x_2 - \mu_2)$$

$$\bar{\Sigma} = \Sigma_{11} + \Sigma_{12}\Sigma_{22}^{-1}\Sigma_{21}.$$

$$\begin{aligned}\mu_{y|x} &= \mathbb{E}_{y|x}[y] = \mathbb{E}[y] + \mathbb{E}[yx^\top] \mathbb{E}[xx^\top]^{-1} (x - \mathbb{E}[x]) \\ &= U^\top (UU^\top + \Psi)^{-1} x \\ \Sigma_{y|x} &= \mathbb{E}[yy^\top] - \mathbb{E}[yx^\top] \mathbb{E}[xx^\top]^{-1} \mathbb{E}[xy^\top] \\ &= I - U^\top (UU^\top + \Psi)^{-1} U \\ \mathbb{E}_{y|x}[yy^\top] &= \mathbb{E}_{y|x}[y] \mathbb{E}_{y|x}[y]^\top + \Sigma_{y|x}\end{aligned}$$

conditional mean and covariance matrix

M-step

optimize parameters

$$Q = \mathbb{E}_{y|x}[\log p(x \mid y)]$$

choose U and Ψ s.t.

is optimal

optimize $U \rightarrow \text{gradient} = 0$

$$\nabla_U Q = 0$$

just derivative

help with matrix manipulations $\rightarrow$ matrix cookbook

$$\nabla_\Psi Q = 0$$

optimize Ψ

derivative with respect to Ψ

- E-step: for all x_i in the data set compute

$$\mathbb{E}_{\mathbf{y}|x_i}[\mathbf{y}] = \mathbf{U}^\top (\mathbf{U}\mathbf{U}^\top + \mathbf{\Psi})^{-1} \mathbf{x}_i$$

$$\mathbb{E}_{\mathbf{y}|x_i}[\mathbf{y}\mathbf{y}^\top] = \mathbb{E}_{\mathbf{y}|x_i}[\mathbf{y}] \mathbb{E}_{\mathbf{y}|x_i}[\mathbf{y}]^\top + \mathbf{I} - \mathbf{U}^\top (\mathbf{U}\mathbf{U}^\top + \mathbf{\Psi})^{-1} \mathbf{U}$$

- M-step: update parameters according to

$$\mathbf{U} = \left(\frac{1}{n} \sum_{i=1}^n \mathbf{x}_i \mathbb{E}_{\mathbf{y}|x_i}[\mathbf{y}]^\top \right) \left(\frac{1}{n} \sum_{i=1}^n \mathbb{E}_{\mathbf{y}|x_i}[\mathbf{y}\mathbf{y}^\top] \right)^{-1}$$

$$\mathbf{\Psi} = \frac{1}{n} \text{diag} \left(\sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^\top - \sum_{i=1}^n \mathbf{x}_i \mathbb{E}_{\mathbf{y}|x_i}[\mathbf{y}]^\top \mathbf{U}^\top \right)$$

inverting large matrices $\rightarrow$ costly
use matrix inversion lemma

- Computing $(\mathbf{U}\mathbf{U}^\top + \mathbf{\Psi})^{-1}$ can be slow if m is large
- Using the matrix inversion lemma we get

$$(\mathbf{U}\mathbf{U}^\top + \mathbf{\Psi})^{-1} = \mathbf{\Psi}^{-1} - \mathbf{\Psi}^{-1} \mathbf{U} (\mathbf{I} + \mathbf{U}^\top \mathbf{\Psi}^{-1} \mathbf{U})^{-1} \mathbf{U}^\top \mathbf{\Psi}^{-1}$$

- Note that inverting $\mathbf{\Psi}$ is cheap since it is diagonal

inverting $\mathbf{\Psi}$ is cheap (1 over diagonal entries) because $\mathbf{\Psi}$ is diagonal

$$(\mathbf{I} + \mathbf{U}^\top \mathbf{\Psi}^{-1} \mathbf{U})$$

missing inverse of

FA	PCA
causes of the data	geometrical abstractions
explain common variances	explain all variance
variance shared	first I with max. variance
scale invariant	not scale invariant
additive noise	no noise
solution not unique	solution unique
model assumptions	no assumptions
solution depends on I	first I unique
projection uses noise	no noise
no ranking	ranked by eigenvalues

FA	ICA
additive noise	no noise
solution not unique	unique up to scale, permutation
assumption: Gauss	assumption: super-Gauss
projection averaged over noise	no noise
solution depends on I	does not depend on I

solution to FA not unique
every orthogonal solution is also solution to FA
rotation doesn't change anything
factors are not ranked

additive noise

linear model, just like ICA

FA is kinda opposite of ICA

FA->assumes everything is gaussian

ICA->tries to minimize gaussianity

FA assumes data is gaussian

varimax rotation

takes FA solution

finds other solution maximizing squared loadings

each factor has either large or small loadings of variable

each variable should be explained by some factor

quartimax rotation
minimizes number of factors for explaining each variables
problem: factors explaining many variables
equimax rotation
compromise between varimax and quartimax
empirical data matrices
should be similar to theoretical means/covariances

06 Scaling and Projection methods

PCA, ICA, factor analysis → linear projection methods
projection into principal/independent components/factors
FA removes noise before projection
but doesn't need to be linear
also called scaling
typically into low-dimensional space, e.g. for visualization or preprocessing

Projection pursuit

~ ICA
projection into least gaussian direction
either supergaussian or subgaussian
supergaussian → default value, some outliers
subgaussian → more modes/classes
nongaussianity with entropy
Gaussian distribution maximizes entropy
==> try to minimize entropy of projection direction
hard to optimize
==> use kurtosis or contrast functions

Multidimensional Scaling

also MDS
projection to low-dimensional space with certain constraints
distances should be kept as much as possible
not completely possible in general (more dimensions of freedom in high-dimensional space)

x_i projected to y_i
 x_j projected to y_j
 distance between x_i and x_j should be similar to distance between y_i and y_j

objective functions R_1, R_2, R_3
 penalize if distances differ

R_1
 Kruskal's measure, penalize large distances
 sum of squares
 divided by constant (distance in high-dim space squared)

R_2
 divide distance by high-dimensional space, then square
 doesn't punish large differences that much
 upweights points that are close together in high-dim space

R_3
 Sammon mapping
 downweight squared distances by high-dim differences
 compromise between errors in small and large distances

$$\mathbf{y}_i = f(\mathbf{x}_i; \mathbf{w})$$

$$\delta_{ij} = \|\mathbf{x}_i - \mathbf{x}_j\|$$

$$d_{ij} = \|\mathbf{y}_i - \mathbf{y}_j\| \quad \text{goal: } d = \delta$$

$$R_1(d, \delta) = \frac{\sum_{i < j} (d_{ij} - \delta_{ij})^2}{\sum_{i < j} \delta_{ij}^2} \propto \sum_{i < j} (d_{ij} - \delta_{ij})^2 \quad \begin{array}{l} \text{"Kruskal's measure"} \\ \text{penalizes large errors} \end{array}$$

$$R_2(d, \delta) = \sum_{i < j} \left(\frac{d_{ij} - \delta_{ij}}{\delta_{ij}} \right)^2 \quad \begin{array}{l} \text{fractional (relative)} \\ \text{errors} \end{array}$$

$$R_3(d, \delta) = \frac{1}{\sum_{i < j} \delta_{ij}} \sum_{i < j} \frac{(d_{ij} - \delta_{ij})^2}{\delta_{ij}} \propto \sum_{i < j} \frac{(d_{ij} - \delta_{ij})^2}{\delta_{ij}} \quad \begin{array}{l} \text{"Sammon mapping"} \\ \text{compromise} \end{array}$$

$\propto$ means factors constant in the parameters $\mathbf{w}$

optimizing objectives

gradient descent
 calculate derivatives

$$\frac{\partial}{\partial \mathbf{y}_k} R_1(d, \delta) = \frac{2}{\sum_{i < j} \delta_{ij}^2} \sum_{j \neq k} (d_{kj} - \delta_{kj}) \frac{\mathbf{y}_k - \mathbf{y}_j}{d_{kj}}$$

$$\frac{\partial}{\partial \mathbf{y}_k} R_2(d, \delta) = 2 \sum_{j \neq k} \frac{d_{kj} - \delta_{kj}}{\delta_{kj}^2} \frac{\mathbf{y}_k - \mathbf{y}_j}{d_{kj}}$$

$$\frac{\partial}{\partial \mathbf{y}_k} R_3(d, \delta) = \frac{2}{\sum_{i < j} \delta_{ij}} \sum_{j \neq k} \frac{d_{kj} - \delta_{kj}}{\delta_{kj}} \frac{\mathbf{y}_k - \mathbf{y}_j}{d_{kj}}$$

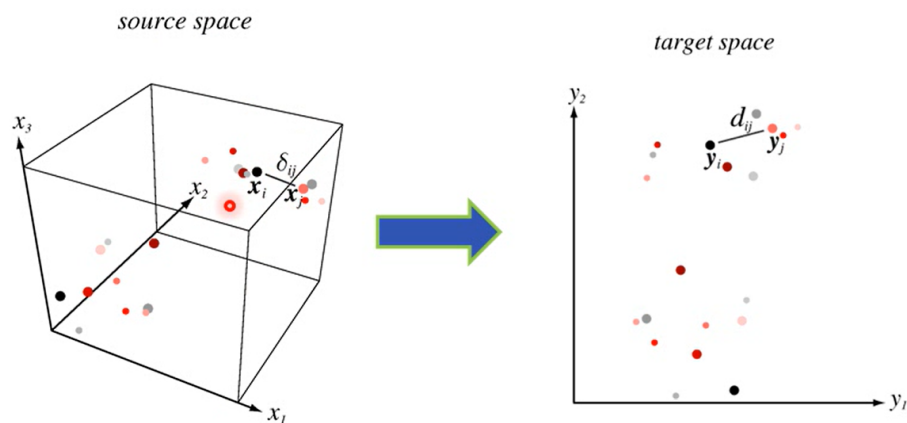
move towards or away depending on whether distance is smaller or larger
like force pulling on y_k

all data points attract or repel data point

like forcefield

size of force is different for different objective functions

find position where all forces summed up equal 0



metric multidimensional scaling (metric MDS)

also “principal coordinates analysis”

linear method, uses euclidian distances

use mean squared solution

Locally linear embedding

nonlinear

try to flatten manifold from high-dimensional space

computes lower-dimensional representation of data points
 wants to preserve neighbourhood
 represent data point by its neighbours

$$\epsilon(\mathbf{W}) = \sum_i \left\| \mathbf{x}_i - \sum_{j=1}^k W_{ij} \mathbf{x}_j \right\|^2$$

close neighbour $\rightarrow$ large weights

constraint: weights sum up to 1

weights describe data point by its neighbours

rotations/translations don't change anything about neighbours

invariant under rotations and translations

local embedding

optimizes downprojection to have similar neighbourhood

least square estimate

similar to PCA

use bottom d eigenvectors except last one

last eigenvector is offset

steps

- select neighbours of datapoint
- describe data point by distances to neighbours
- map to lower-dimensional space, try to conserve neighbourhood distances

algorithm

Given: $\mathbf{X}$: n by m matrix consisting of n data items in m dimensions, dimension of embedding space l , k number of neighbors, distance measure

Find neighbors in $\mathbf{X}$ space

```
for ( $i = 1$  ;  $i \leq n$  ;  $i++$ ) do
    compute the distance from  $\mathbf{x}_i$  to every other point  $\mathbf{x}_j$ 
    find the  $k$  smallest distances
    assign the corresponding points to be neighbors of  $\mathbf{x}_i$ 
end for
```

Solve for reconstruction weights $\mathbf{W}$

```
for ( $i = 1$  ;  $i \leq n$  ;  $i++$ ) do
    create matrix  $\mathbf{Z}$  consisting of all neighbors of  $\mathbf{x}_i$  [d]
    subtract  $\mathbf{x}_i$  from every row of  $\mathbf{Z}$ 
    compute the local covariance  $\mathbf{C} = \mathbf{Z}^T \mathbf{Z}$  [e]
    solve linear system  $\mathbf{C}\mathbf{w} = \mathbf{1}$  for  $\mathbf{w}$  [f]
    set  $W_{ij} = 0$  if  $j$  is not a neighbor of  $i$ 
    set the remaining elements in the  $i$ -th row of  $\mathbf{W}$  equal to  $\mathbf{w} / \sum_j (w_j)$ ;
end for
```

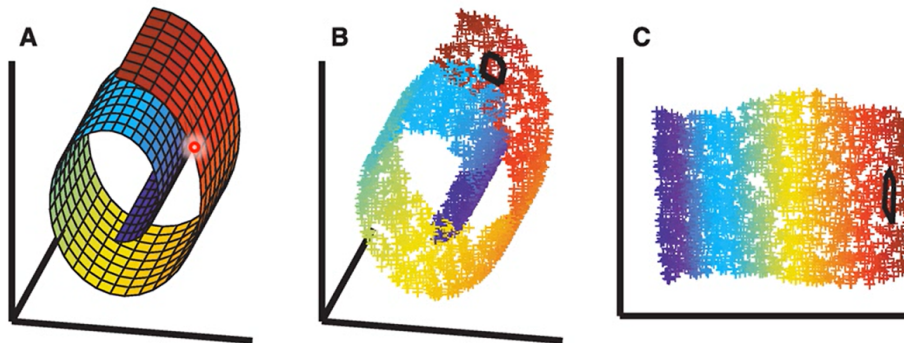
Compute embedding coordinates $\mathbf{Y}$ using weights $\mathbf{W}$

```
create sparse matrix  $\mathbf{M} = (\mathbf{I} - \mathbf{W})^T (\mathbf{I} - \mathbf{W})$ 
find bottom  $l + 1$  eigenvectors of  $\mathbf{M}$  (corresponding to the  $d + 1$  smallest eigenvalues)
set the  $q$ -th column of  $\mathbf{Y}$  to be the  $q + 1$  smallest eigenvector (discard the bottom eigenvector  $\mathbf{1} = (1, 1, 1, \dots)$  with eigenvalue zero)
```

Result $\mathbf{Y}$: n by l matrix consisting of $l < m$ dimensional embedding coordinates.

solve small linear equation system multiple times, eigendecomposition

LLE for Swiss Roll data



goal: unrolling

LLE $\rightarrow$ always looks at local neighbours

LLE doesn't work well if there are overlaps/noise

isomap

related to LLE/same idea

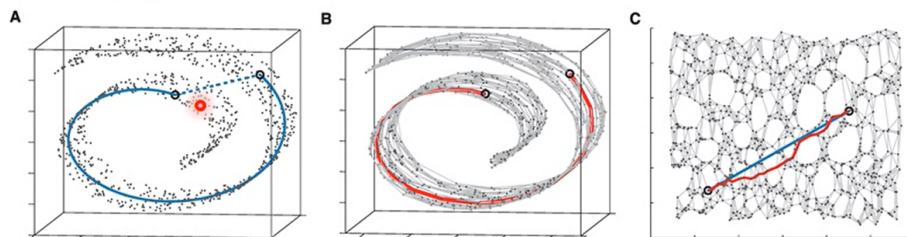
try to flatten manifold with local neighbours

computes quasi-isometric low-dimensional embedding

use geodesic distance

go along manifold, get distance along manifold

shortest distance along manifold



view data points as graph, walk along graph, shortest path along graph

e.g. dijkstra

use largest eigenvectors of geodesic distance matrix

algorithm

doubly centered geodesic distance matrix $\tau(\mathbf{D})$

$$\tau(\mathbf{D}) = -\frac{1}{2} \mathbf{H} \mathbf{D}^2 \mathbf{H}$$

where $\mathbf{D}^2 = D_{ij}^2 = D_{ji}^2$ is the element-wise square of the geodesic distance matrix

$\mathbf{H}$ is the centering matrix $\mathbf{H} = \mathbf{I}_n - \frac{1}{n} \mathbf{1} \mathbf{1}^T$ $\mathbf{1} = (1, 1, \dots, 1)^T \in \mathbb{R}^n$

objective of Isomap: $\mathbf{E} = \|\tau(\mathbf{D}_X) - \tau(\mathbf{D}_Y)\|_{L^2}$

$\mathbf{D}_Y$ matrix of Euclidean distances in the projected space

$\mathbf{D}_X$ matrix of geodesic distances

τ converts distances to inner products (squared distances)

The objective can be minimized by setting the coordinates $\mathbf{y}_i$ to the top l eigenvectors of the matrix $\tau(\mathbf{D}_X)$

distance metric produced by finding all neighbours and all distances (Dijkstra)

center distance matrix

want euclidian distances in low-dim space to be same as geodesic distances in high-dim space

minimize objective by finding ideal coordinates

Given: distances $d(\mathbf{x}_i, \mathbf{x}_j)$ between pairs from n data points in an m -dimensional space X , parameter k or parameter e

Construct neighborhood graph
Define the graph G over all data points by connecting points $\mathbf{x}_i$ and $\mathbf{x}_j$ if they are closer than e (e -Isomap), or if i is one of the k nearest neighbors of j (k -Isomap). Closeness and neighborhood is measured by $d(\mathbf{x}_i, \mathbf{x}_j)$.
Set edge lengths equal to $d(\mathbf{x}_i, \mathbf{x}_j)$.

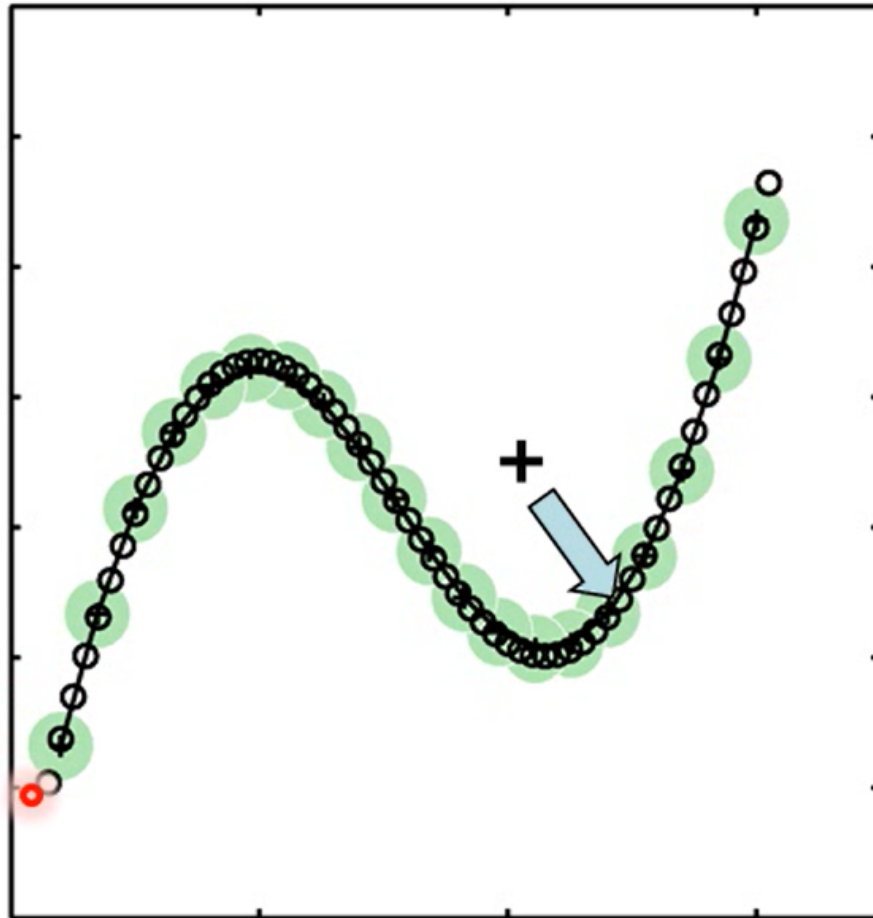
Compute shortest paths by Floyd's algorithm
for ($i = 1 ; i \leq n ; i++$) **do**
 for ($j = 1 ; j \leq n ; j++$) **do**
 Initialize $d_G(\mathbf{x}_i, \mathbf{x}_j) = d(\mathbf{x}_i, \mathbf{x}_j)$ if i, j are linked by an edge; $d_G(\mathbf{x}_i, \mathbf{x}_j) = \infty$, otherwise.
 end for
end for
for ($k = 1 ; k \leq n ; k++$) **do**
 for ($i = 1 ; i \leq n ; i++$) **do**
 for ($j = 1 ; j \leq n ; j++$) **do**
 $d_G(\mathbf{x}_i, \mathbf{x}_j) = \min\{d_G(\mathbf{x}_i, \mathbf{x}_j), d_G(\mathbf{x}_i, \mathbf{x}_k) + d_G(\mathbf{x}_k, \mathbf{x}_j)\}$.
 end for
 end for
end for
define shortest path matrix D_X by $[D_X]_{ij} = d_G(\mathbf{x}_i, \mathbf{x}_j)$

Construct l -dimensional embedding
Compute λ_p as the p -th eigenvalue (in decreasing order) of the matrix $\tau(D_X)$,
and v_{pi} as the i -th component of the p -th eigenvector.
set $y_{ij} = \sqrt{\lambda_i} v_{ji}$.
Result Y : coordinate vectors $\mathbf{y}_i$ in a l -dimensional ($l < m$) Euclidean space
 Y

use isomap of k neighbours or ϵ describing distance to be a neighbour
requires many datapoints with few noise/high-populated manifold

Generative topographic mapping

generative model
similar to FA
latent variables mapped to observations
latent variables in low-dim space
observations in high-dim space
assume observations are center of gauss ball/curve
generate/calculate distribution over all centers
log likelihood
after learning, all gauss balls are around data points



t-distributed stochastic neighbour embedding (t-SNE)

often used for presenting ML results

maps high-dim observations to lower-dimensional (typically 2D or 3D) space

used for visualization, finding clusters/structures etc

based on SNE (stochastic neighbour embedding)

compute similarity with conditional probability

how likely is data point generated with gauss distribution with that center?

normalize over all gauss clusters

re-evaluate clusters after finding data point

calculate conditional probabilities in high- and low-dim space

compare
try to make them equal/similar
want Kullback-Leibler divergence between probabilities
minimize with gradient descent

problems

- difficult to optimize
- crowding problem
 - assume high-dim space
 - data points with equal distance to each other
 - distances between pairs of data points cannot be the same
 - ==> data points are crowded together in one place
 - clusters pushed close to each other, overlaying, etc.

t-SNE
==>use Student's t-distribution

advantages

- objective is symmetric
 - simpler gradients, simpler optimization
- t-distribution results in better separation of clusters
 - t-distribution allows outliers etc

symmetrize conditional probabilities

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$$

symmetric similarity matrix

self-organizing maps

clustering with other nice properties
also Kohonen maps
clustering + downprojection
problem: don't have objective function/error function
no theoretical basis for choosing parameters
cannot ensure topographic ordering
don't know whether converging/when to stop
cannot compare model (which result is better)
cannot detect overfitting
don't know how to improve method
hard to check quality of solution

y -s have neighbourhood relation

start with randomly initialized w s

update rule

$$k = \arg \max_s x^\top w_s$$

set k to largest dot product

pick w most similar to x

$$(w_t)^{new} = w_t + \eta \delta (||y_t - y_k||)(x - w_t)$$

push w towards x

η learning rate

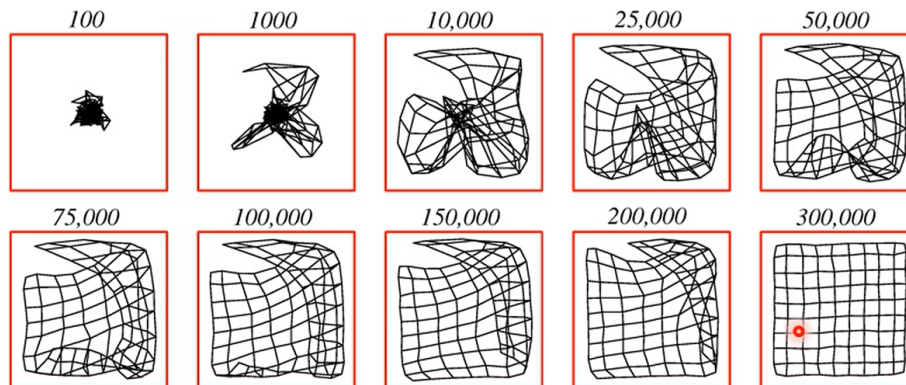
neighbours move nearer

window function decides how much it is pushed

neighbours are computed in low-dim space

distance $\rightarrow$ check distance to closest w

(steps)



can get stuck in local minima

cannot distinguish good and bad solutions (in high-dim space)

can reflect density

choose all data points equally

Non-Negative Matrix factorization

Matrix decomposition method

all entries of matrix must be non-negative

part-based representation

only describe what is seen but not what's missing

described parts only contribute positively

data matrix X is positive

$$X = YU^\top$$

X, Y, U all non-negative

X -> which parts are present in which data samples

Y -> actual parts where data is present

U -> how parts are reflected in data

measurement matrix X

internal dimension l

smaller than n and m

low-rank decomposition, middle dimension smaller

update rules

2 basic algorithms, different objectives

both objective describe how distant $YU^\top$ is from X

Kullback-Leibler divergence

sum up matrix, should add up to 1

then can be viewed as multinomial distribution

Minimize Kullback-Leibler divergence $D(X||YU^\top)$

using gradient descent

$$Y_{ik} = Y_{ik} \frac{\sum_{j=1}^m U_{jk} X_{ij} / (Y U^\top)_{ij}}{\sum_{j=1}^m U_{jk}}$$

$$U_{jk} = U_{jk} \frac{\sum_{i=1}^n Y_{ik} X_{ij} / (Y U^\top)_{ij}}{\sum_{i=1}^n Y_{ik}}$$

update rule has fixed point

additive update

Euclidian distance

Frobenius Norm

$$\|X - YU^\top\|_F^2$$

$$\|A - B\|_F^2 = \sum_{ij} (A_{ij} - B_{ij})^2$$

also gradient descent

$$Y_{ik} = Y_{ik} \frac{(X U)_{ik}}{(Y U^T U)_{ik}} \quad \begin{array}{l} \text{multiply } \mathbf{X} = \mathbf{Y} \mathbf{U}^T \\ \text{from right by } \mathbf{U} \end{array}$$

$$U_{jk} = U_{jk} \frac{(Y^T X)_{kj}}{(Y^T Y U^T)_{kj}} \quad \begin{array}{l} \text{multiply } \mathbf{X} = \mathbf{Y} \mathbf{U}^T \\ \text{from left by } \mathbf{Y}^T \end{array}$$

also has fixed point

left and right side is equal

multiplicative update, faster than additive update

can be extended to sparse matrices

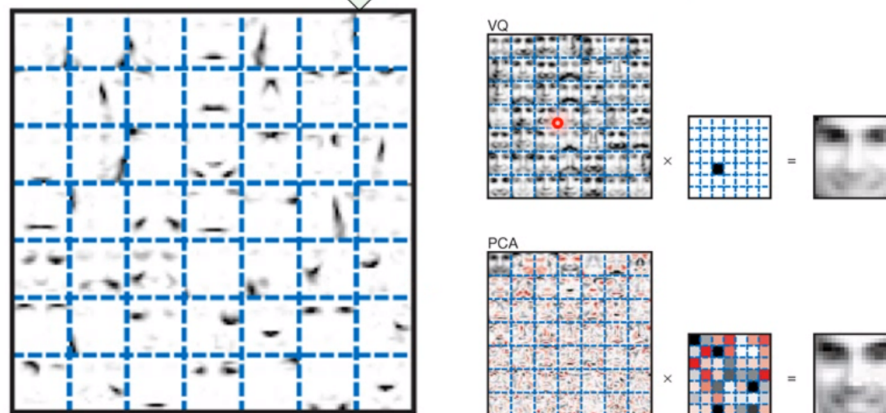
both matrices can be sparse

Y sparse $\rightarrow$ few parts are present

U sparse $\rightarrow$ parts only effect few measurements

parts-based representations of faces

vector quantization (VQ)
and PCA learn
holistic representations



matrix with possible part, other matrix for selecting parts
(VQ $\rightarrow$ vector quantization)

PCA

just largest variance

no interpretation of components

07 Clustering

regions with high data density grouped together
may correspond to prototypical example + noise

visualization→important application

observation represented by prototypes (e.g. cluster centers)
“vector quantization”
vector represented by its prototype

Mixture models

most important clustering method
generative models

components→description of mixture model

component j out of l components has params

location parameter μ_j

width/shape param Σ_j

weight w_j

how much data is explained by that component

w_j

probability $p(j)$ of choosing component j

with probability density $p(x|j, \theta_j)$

θ_j summarizes params of component j

Θ summarizes all params into big vector

$$p(x|\Theta) = \sum_{j=1}^l p(j)p(x|j, \theta_j)$$

Bayes rule for clustering

$$p(j|x, \theta) = \frac{p(x|j, \theta_j)p(j)}{p(x|\theta)}$$

observation is assigned to component with largest posterior (left side)

prior probability $p(j)$

probability without observation

component generates data point

different mixture components

gaussians, Poisson, ...

$p(x|j, \theta_j)$ is distributed like that

optimizing

using log likelihood
e.g. gradient descend

posterior expectation (expected value) of derivative of log-likelihood of component j

Gaussian mixture

all components are gaussian distribution
params: mean and covariance matrix

$$\theta_j = (\mu_j, \Sigma_j)$$

expectation maximization

EM-algorithm:

E-step:

$$p(j \mid \mathbf{x}_i, \mu_j, \Sigma_j) = \frac{w_j \mathcal{N}(\mu_j, \Sigma_j)(\mathbf{x}_i)}{\sum_{t=1}^l w_t \mathcal{N}(\mu_t, \Sigma_t)(\mathbf{x}_i)}$$

M-step:

$$\begin{aligned} w_j^{\text{new}} &= \frac{1}{n} \sum_{i=1}^n p(j \mid \mathbf{x}_i, \mu_j, \Sigma_j) \\ \mu_j^{\text{new}} &= \frac{\sum_{i=1}^n p(j \mid \mathbf{x}_i, \mu_j, \Sigma_j) \mathbf{x}_i}{\sum_{i=1}^n p(j \mid \mathbf{x}_i, \mu_j, \Sigma_j)} \\ \Sigma_j^{\text{new}} &= \frac{\sum_{i=1}^n p(j \mid \mathbf{x}_i, \mu_j, \Sigma_j) (\mathbf{x}_i - \mu_j) (\mathbf{x}_i - \mu_j)^T}{\sum_{i=1}^n p(j \mid \mathbf{x}_i, \mu_j, \Sigma_j)} \end{aligned}$$

done per component

E

calculate posterior probability

M

calculate new w , μ (weighted sum) and Σ

can be extended to maximum posterior algorithm

maximize posterior probability

include prior distribution for parameters

Wichart density for prior Σ

Dirichlet density for prior weights

gaussian density for prior μ

also EM

$$\alpha = \frac{m}{2}$$

$$\Psi = \frac{1}{2}\mathbf{I} \quad \text{OR} \quad \Psi = \frac{1}{2}\text{covar}(\mathbf{x})$$

$$\gamma = 1$$

$$\eta = 0$$

$$\nu_j = \text{mean}(\mathbf{x})$$

second Ψ -> covariance estimated

component could be sitting on single data point

can result in really high density due to a point very similar to it
overfitting

==> sigmas should have eigenvalues 0

data points could belong to multiple clusters

mixture of Poissons

Poisson -> discrete distribution

good for data where something is counted

e.g. count of accidents

alternatives

binomial and negative binomial distribution

variance and mean is the same (λ)

negative binomial

variance larger than mean

binomial

variance lower than mean

probability mass function (not density)

$$P(x, \lambda) = \frac{1}{x!} e^{-\lambda} \lambda^x$$

l components, just 2 parameter vectors
weights and lambdas

$$p(x) = \sum_{j=1}^l w_j P(x; \lambda_j)$$

posterior probability

$$p(\mathbf{w}, \boldsymbol{\lambda} \mid x) = \frac{p(x \mid \mathbf{w}, \boldsymbol{\lambda}) p(\mathbf{w}) p(\boldsymbol{\lambda})}{\int p(x \mid \mathbf{w}, \boldsymbol{\lambda}) p(\mathbf{w}) p(\boldsymbol{\lambda}) d\mathbf{w} d\boldsymbol{\lambda}}$$

initialization

EM (expectation maximization)

- multiple low tolerance runs

- fast

- then more precise slow run

rnd

- multiple random initializations, pick best

svd

- use SVD on data matrix

- spectral clustering

possible constraints of params

- spherical

- covariance must be multiple of identity matrix

- all components have same variance

- diagonal

- clusters can only be squeezed/stretched along axis

- volume

- all clusters must have same volume (weighting factor)

constraints make mixture of gaussians more robust/reduce complexity

volume can be combined with others

also equal shape component possible (redundant if spherical)

constraints can be put on mixture or single components

k-means

simple, fast and robust version of mixture clustering

- equal weight for each component

- spherical covariance matrix

- equal between components

covariance matrix is identity matrix
discrete cluster membership (data point belongs to cluster or doesn't)

update → center cluster centers

start with random cluster centers
iterate until cluster centers don't change
 check which data point which cluster belongs to
 update center points as center of clusters

properties

- fast
- robust against outliers
- simple
- affected by initialization

fuzzy k-means

soft memberships
“probability” how much data point belongs to cluster
 ~ softmax

update rule

$$\mu_j^{\text{new}} = \frac{\sum_{i=1}^n p^b(j | \mathbf{x}_i, \mu_j) \mathbf{x}_i}{\sum_{i=1}^n p^b(j | \mathbf{x}_i, \mu_j)}$$

The following objective is minimized:

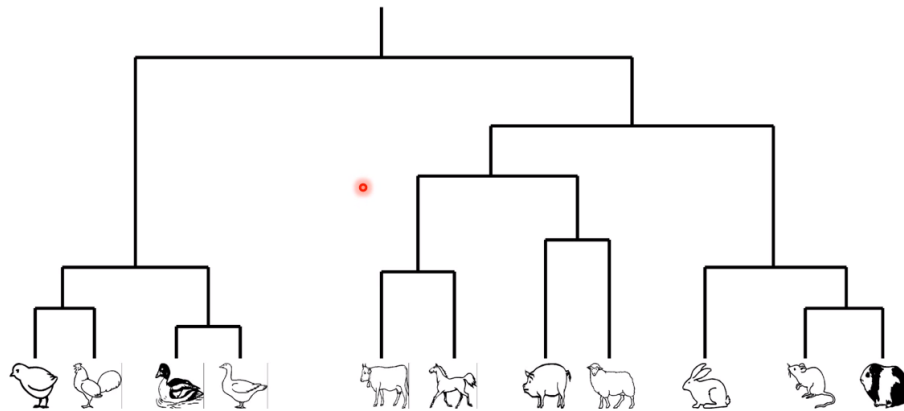
$$\sum_{j=1}^l \sum_{i=1}^n p^b(j | \mathbf{x}_i, \mu_j) \|\mathbf{x}_i - \mu_j\|^2$$

This algorithm is called **fuzzy k -means clustering**

Hierarchical clustering

clusters as trees

bottom up (agglomerative) or top-down (divisive)



bottom-up==>start with 1-element clusters, merge
 needs distance measurement
 can calculate between single examples
 distance between clusters
 single-link->min distance between two data points in different clusters
 clusters cannot overlay
 complete link->max distance -"
 avoids clusters including elements far away from their cluster center
 but may not separate clusters well (could be overlaying etc)
 average link 1->mean of all distance combinations
 very common
 "unweighted pair group method using arithmetic averages"
 average link 2->calculate cluster centers, mean
 top-down==>start with single cluster, split
 often makes use of graph theory
 compute minimal spanning tree
 remove largest edge
 ==>divided in two subtrees

similarity-based clustering

other clustering methods based on distances between data points
 can instead use similarities between data points
 often only similarities between objects but not other information (e.g. don't have feature vectors)

similarities could be

- links in webpages
- interactions of humans (e.g. social media)
- co-occurrences of objects
 where do objects occur together? e.g. co-expression of genes

- or papers citing each other
- or about same topic
- spatial distances
- co-processing
 - e.g. documents
- sequence alignment
- structural alignment

aspect model

consider discrete data of observations

model:

$$p(x, y) = \sum_z p(z) p(x|z) p(y|z)$$

hidden variable z

assumes that x and y are independent of each other but but conditioned on z

$$p(x, y) = \sum_z p(x, y, z) = \sum_z p(z) p(x, y|z) = \sum_z p(z) p(x|z) p(y|z)$$

hidden factor z

effect on both x and y

“Consumption of hot chocolate is correlated with low crime rate, but both are responses to cold weather.”

- x = consumption of hot chocolate
- y = crime rate
- z = cold weather

can use EM for optimizing maximum likelihood

z indicates cluster

each z represents cluster

affinity propagation

exemplar (prototype)-based

exemplars are existing real examples

gives already observed prototypes

comes from message propagation

k-centers

start with randomly selected exemplars, iterative

only for few clusters

requires good starting exemplars

given

similarities between object i and k : $s(i, k)$

preferences $s(k, k)$

hyperparameter, how likely object k becomes an exemplar
responsibilities $r(i, k)$
messages sent from i to candidate exemplar k
reflect evidence that k serves as exemplar for i
how well k explains/represents i
availabilities
messages sent from candidate exemplar k to i
Would cluster already explain many elements?
Is candidate available for i ?

initialization: $a(i, k) = 0$

updates:

$$r(i, k) = s(i, k) - \max_{k', k' \neq k} \{a(i, k') + s(i, k')\}$$

$$a(i, k) = \min \left\{ 0, r(k, k) + \sum_{i', i' \notin \{i, k\}} \max\{0, r(i', k)\} \right\}$$

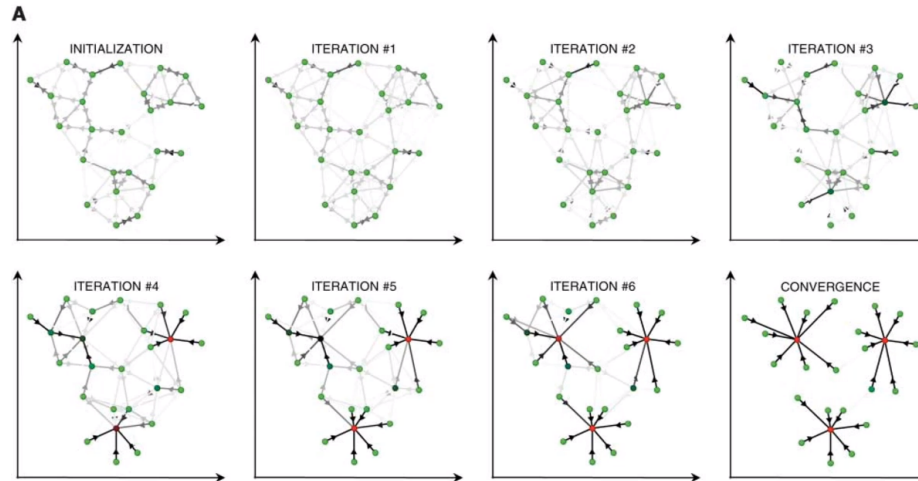
$$a(k, k) = \sum_{i', i' \neq k} \max\{0, r(i', k)\}$$

r - low if other candidates are “better”

$a(i, k)$ - low availabilities if many other items also want this as an exemplar or if item is very self-similar

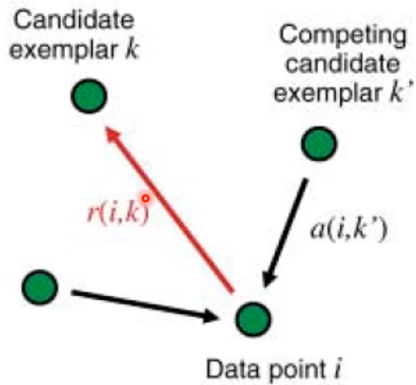
$a(k, k)$ self-availability $\rightarrow$ “How many elements am I responsible except myself?”

iteratively until convergence

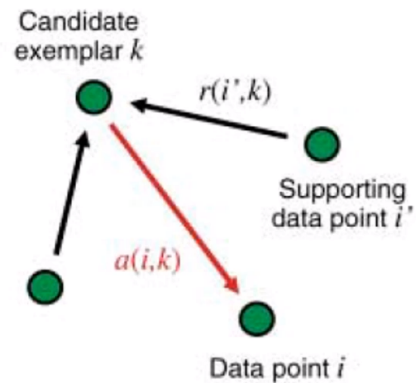


with message passing

Sending responsibilities



Sending availabilities



assumes sphered equal large clusters
difficulties in detecting clusters within clusters

similarity-based mixture models

using mixture models for similarity-based clustering
e.g. SNE uses probabilities
transform similarities to output of mixture models

optimize negative log posterior

covariance matrix can also be represented by similarities
more complicated/inefficient (matrix inversion etc)

can work with clusters of different densities/overlapping clusters etc
can adjust to very small or large clusters

08 Biclustering

simultaneously clusters rows and columns

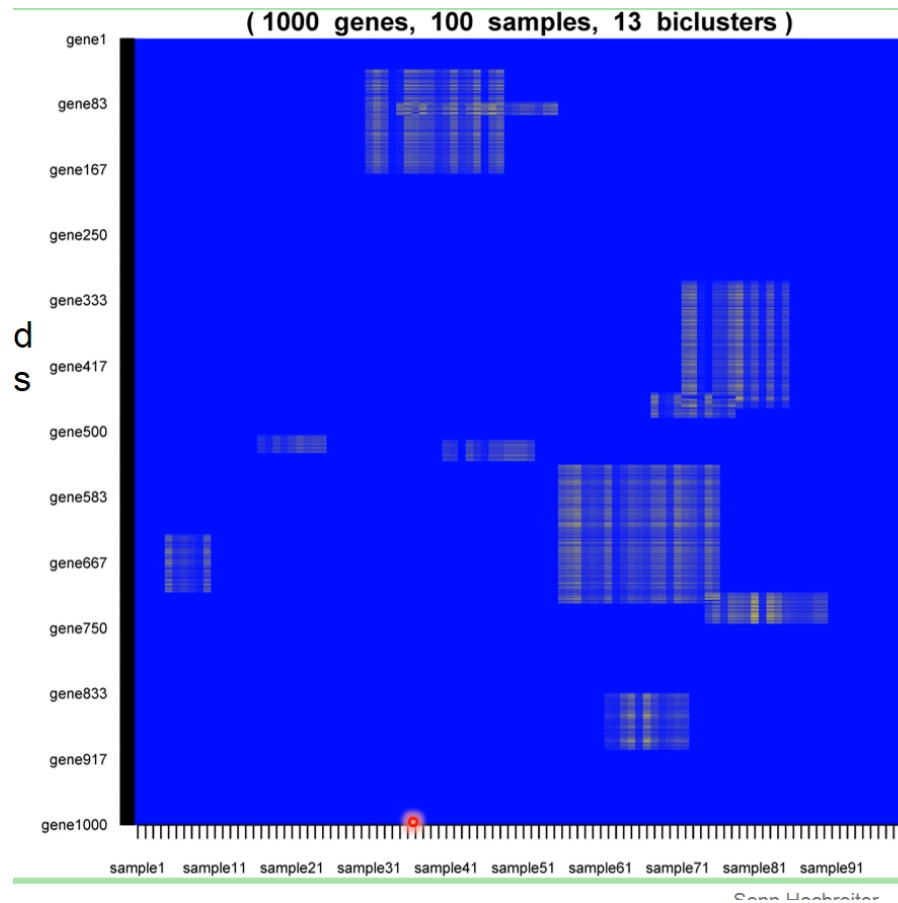
bicluster

pair

set of row elements and set of column elements
row elements in same cluster similar to each other on row elements
same with column elements

element (row+column) can also belong to multiple or no bicluster at all

row elements clustered on subgroup of column elements



block is bicluster
 biclusters can overlay
 can overlay in one dimension or multiple

types of biclusters

- constant values
 all values have same values
- constant row values
 same value in each row
- constant column values
 same value for each column
- additive coherent bicluster
 difference between two elements is always the same

1.0	4.0	5.0	0.0	1.5
4.0	7.0	8.0	3.0	4.5
3.0	6.0	7.0	2.0	3.5
5.0	8.0	9.0	4.0	5.5
2.0	5.0	6.0	1.0	2.5

coherent values: additive

can get other row by adding constant to row
 can also be multiplicative or columnwise
 multiplicative coherent values → can get other row by multiplying row
 with constant
 general coherent values
 order is the same for each column

biclustering methods

variance minimization methods

for constant biclusters

define clusters as blocks in matrix

should deviate minimally from elements

e.g. max deviation δ

two-way clustering

first cluster row elements according to columns

then cluster columns according to rows

apply conventional clustering to columns and rows

can be done iteratively

motif and pattern recognition methods

e.g. row is multiplicative of another row

biclusters share common pattern or motif

probabilistic and generative approaches

“Can I explain the observed matrix as constructed by biclusters”

using model-based techniques

generative models, explain data

e.g. FABIA (Factor analysis for bicluster acquisition)

e.g. BBC (Bayesian BiClustering model)

BBC/FABIA → generative

- model selection well-understood (maximum likelihood)
- hyperparameter selection can be done
- signal-to-noise ratios (can explain noise)
- different models can be compared with likelihood or posterior probability
- tests like likelihood ratio test can be done
- global model for explaining all data

FABIA

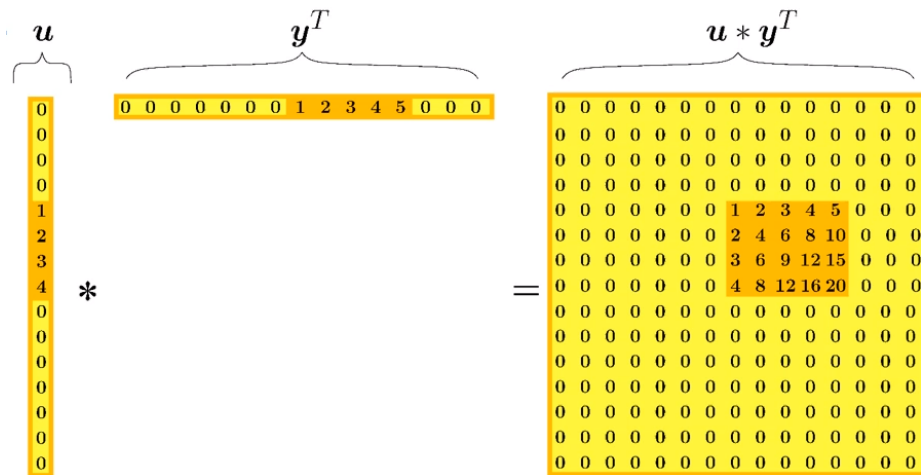
developed by ML institute for analysing genes

biclusters are outer product uy^T

u → prototype column vector

not participating features are 0

y → factors



model for generating l biclusters and additive noise

$$X = \sum_{j=1}^l u_j y_j^T + \Upsilon = YU^T + \Upsilon$$

matrix decomposition

u and y vectors are very sparse

goal: explain matrix X by sparse matrices and a bit of noise

rewrite as

$$x_i = \sum_{j=1}^l u_j y_{ij} + \epsilon_i = U \tilde{y}_i + \epsilon_i$$

i -th row of factor matrix:

$$\tilde{y}_i = (y_{i1}, \dots)$$

basically factor analysis model

but want sparse factors and sparse loading matrix

assumes noise independent of factors
 y and ϵ are normally distributed
 factor loading matrix $\psi \in \mathbb{R}^{l \times l}$ diagonal

need to include sparsity

2 prior probabilities

parameter prior probability

laplace distribution

most probable value is 0

factor prior probability

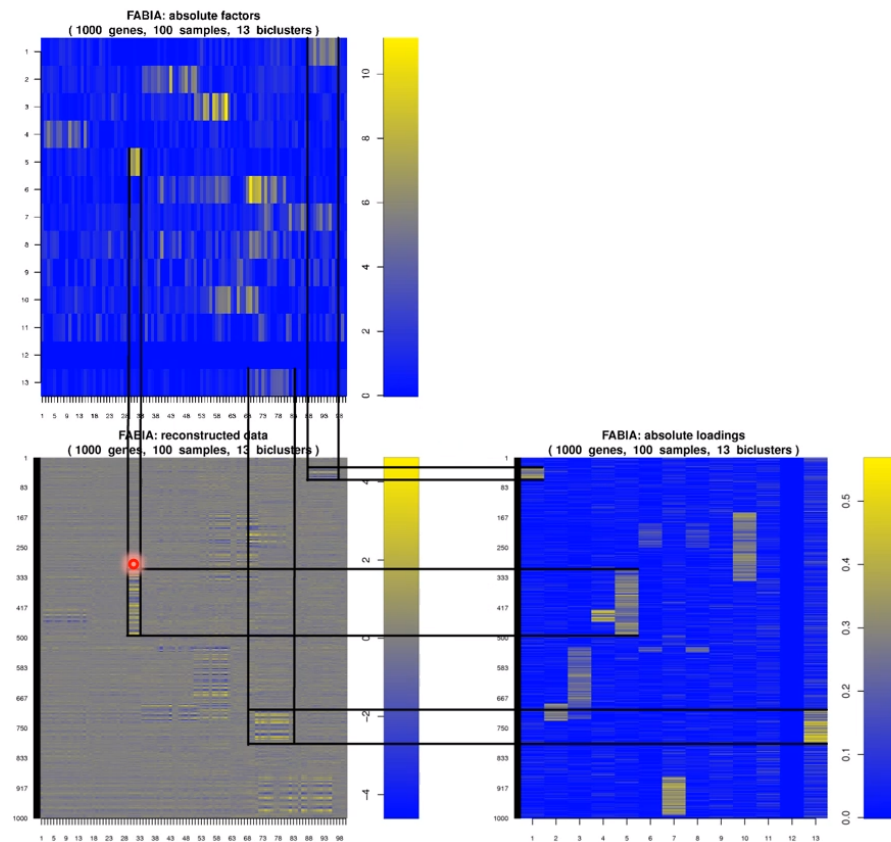
also laplace distribution

likely 0

cannot compute likelihood because of Laplace distribution

EM doesn't work

but variant of EM==>variational expectation maximization



number of biclusters<->number of “hidden factors”
 only few biclusters

first biclusters → explain most of data
 can create more biclusters explaining more data

